Guide Questions

1. Data Structure Selection & Trade-Offs: Why are dynamic arrays preferred over fixed-size static arrays for unpredictable batch workloads, and what hidden computational costs occur during memory reallocations?

   Dynamic arrays are better for unpredictable workloads because they can grow when more data is added. Fixed size arrayshave a set limit, which can lead to wasted memory or not having enough space. The downside is that when a dynamic array grows, the system may beed to allocate new memory and copy the existing elements, which takes extra time and memory

2. Memory Hygiene & Lifecycle: Explain the role of the C++ destructor (~DynamicArray()) in managing heap allocation safety. What system failure occurs if delete[] is omitted in long-running C++ applications?

   The destructor DynamicArray() makes sure the memory used by the array is properly released when the object is destroyed. If delete[] is forgotten, a memory leak occurs. In long running programs, these leaks can build up and eventually cause slow performance, memory exhaustion, or crashes

3. Defensive Programming & Bounds Enforcement: Contrast the behavior of raw array indexing (array[i]) with bounds-checked operations. Why is missing upper-bound validation (idx >= size) dangerous in security-critical codebases?

   Raw array indexing like array[i] does not automatically check if the index is valid. If I is outside the arrays range, The program may access the wrong memory. Missing a check like idx >= size is dangerous because it can cause data corruption, crashes, or security vulnerabilities. Bounds checking helps prevent these problems

4. Algorithmic Overhead & Code Smells: How did eliminating redundant nested loops inside the array deletion routine change the runtime performance of the system?

   Removing the unnecessary nested loops made the deletion process more efficient, Instead of doing extra work repeatedly, the program now performs only the operations it actually needs, This improves runtime, especially when working with larger arrays

5. Value of Peer Code Audits: Why are structured peer code reviews essential in commercial software development prior to merging feature branches into production repositories?

   Peer code reviews are important because another developer can spot problems that the original programmer may have missed. Reviews can catch bugs, memory issues, security risks, and inefficient code before it reaches production. This makes the software more reliable and easier to maintain

REFLECTION QUESTIONS

1. Experiential Reflection: What was the most challenging defect to locate inside the legacy codebase, and how did systematic unit testing assist you in pinpointing the root cause compared to random debugging?

   The most challenging defect was finding the issue hidden in the existing array logic. Systematic unit testing made it easier because I could test specific functions and conditions one at a time. This helped narrow down the exact cause instead of guessing and changing random parts of the code

2. Architectural Evaluation: Knowing what you now know about dynamic array reallocations and shift-based element deletions, what linear architecture alternative (e.g., Linked Lists, Queues,Stacks) would you recommend for systems that require frequent mid-list insertions without reallocating buffers?

   For systems that need frequent insertions in the middle of a list, I would recommend a linked list. Unlike dynamic arrays, linked lists do not need to reallocate or shift many elements when inserting. This makes them more suitable for applications where frequent mid list insertions are required

3. Professional Growth: How has evaluating another programmer's code during the Peer Code Review altered your approach to writing clean, well-documented C++ code for your upcoming Milestone 1 project?

   Reviewing another programmers code showed me the importance of writing code that is clear and easy to understand. I will focus more on proper naming, error, handling, and keeping our code organized. I also learned that writing clean code makes it much easier for others to review and maintain.